

Escalation System

CTP Job Cards — Notification Escalation System

Engineering Reference

Contents

1. [The Big Picture](#)
 2. [The Two Players](#)
 3. [The Firestore Config Document](#)
 4. [Recipient Rules](#)
 5. [The Escalation Lifecycle](#)
 6. [The Stamp Fields](#)
 7. [The Admin Settings UI](#)
 8. [Common Tasks](#)
 9. [Troubleshooting](#)
 10. [File Map](#)
 11. [Hard Rules — Don't Break These](#)
-

1. The Big Picture

When a job card is created, technicians get notified immediately. If nobody responds within a set time, the system **escalates** — sending follow-up notifications to wider groups of people (managers, foremen, etc.) at predefined intervals.

The escalation has **4 stages**, each with its own:

- **Timing** — how many minutes after the job was created
- **Enabled flag** — can be turned on/off independently
- **Recipients per job type** — different people can be notified for mechanical vs electrical jobs

Stage	Timing	Purpose
Stage 1	5 min	First escalation — onsite managers + foremen
Stage 2	10 min	Dept managers + workshop manager
Stage 3	disabled	Reserved for offsite managers
Stage 4	disabled	Final escalation stage

Everything is **config-driven** from a single Firestore document, so changes to who-gets-notified-when don't need a code deployment.

2. The Two Players

A. Firestore document: `notification_configs/global`

The single source of truth for who gets notified and when. Edit this document directly, or use the Admin screen to change escalation behaviour.

B. Cloud Function: `escalateNotifications`

A scheduled function that runs **every 2 minutes** in `europa-west1`. It reads the config, finds eligible jobs, and sends escalation notifications.

Warning — Don't move the schedulers. `escalateNotifications` and `autoCloseMonitoringJobs` must stay in `europa-west1`. Don't change them to `africa-south1` even though the rest of the project is there — this will break the Cloud Scheduler binding.

3. The Firestore Config Document

```
{  
  "stages": {
```

```

"stage1": {
  "enabled": true,
  "enabled_at": "2026-05-15T17:00:00.000Z",
  "minutes": 5,
  "recipients_by_type": {
    "mechanical": ["onsite_managers", "foremen"],
    "electrical": ["onsite_managers", "foremen"],
    "mech/elec": ["onsite_managers", "foremen"]
  }
},
"stage2": { ... },
"stage3": { ... },
"stage4": { ... }
},

"creation_recipients_by_type": {
  "mechanical": ["onsite_mechanics"],
  "electrical": ["onsite_electricians"],
  "mech/elec": ["onsite_mechanics", "onsite_electricians"]
},

"excluded_job_types": ["maintenance"],

"last_updated": "2026-05-15T17:00:00Z",
"updated_by_clock_no": "22"
}

```

Field-by-field

Field	Purpose
<code>stages.stageN.enabled</code>	If <code>false</code> , that stage is completely skipped — no query, no notifications, no stamps written
<code>stages.stageN.enabled_at</code>	ISO-8601 timestamp set when the stage transitions <code>disabled</code> → <code>enabled</code> . Jobs created before this moment are skipped — prevents flooding newly-added recipients with notifications for old open jobs. <code>null</code> means no filter (legacy or always-on stages).

Field	Purpose
<code>stages.stageN.minutes</code>	How many minutes after <code>createdAt</code> the stage triggers. Must be > 0
<code>stages.stageN.recipients_by_type</code>	Map of job type → list of recipient rule names
<code>creation_recipients_by_type</code>	Who gets notified the instant a job card is created (before any escalation)
<code>excluded_job_types</code>	Job types that should never escalate (e.g. internal maintenance)
<code>last_updated</code> / <code>updated_by_clock_no</code>	Audit trail of who last changed the config

Job type keys

Dart enum	Firestore key
<code>mechanical</code>	<code>"mechanical"</code>
<code>electrical</code>	<code>"electrical"</code>
<code>mechanicalElectrical</code>	<code>"mech/elec"</code> (note the slash)
<code>maintenance</code>	listed in <code>excluded_job_types</code>

4. Recipient Rules

Each rule resolves to a list of employees from the `employees` Firestore collection at runtime. Rules are evaluated **per job** — the same rule returns different people for a mechanical job vs an electrical job (because the rule itself is type-aware).

Rule	Who it returns
<code>operator</code>	The job creator (looked up via <code>operatorClockNo</code> on the job). Special case: ignores <code>isOnSite</code> and receives a tailored notification — see callout below.

Rule	Who it returns
<code>onsite_mechanics</code>	Employees with <code>isOnSite: true</code> and position containing <code>mechanic / mechanical</code> (not manager)
<code>onsite_electricians</code>	Employees with <code>isOnSite: true</code> and position containing <code>electrician / electrical</code> (not manager)
<code>onsite_managers</code>	Onsite managers in department matching the job type (e.g. mechanical jobs → Mechanical or Workshop dept managers)
<code>foremen</code>	Onsite employees in the job's department with position containing <code>foreman</code> or <code>shift leader</code>
<code>onsite_dept_managers</code>	Onsite managers in the job's specific department
<code>onsite_workshop_manager</code>	The onsite Workshop department manager (excluding mech/elec-specific managers)
<code>offsite_managers</code>	Same as <code>onsite_managers</code> but with <code>isOnSite: false</code>
<code>offsite_dept_managers</code>	Same as <code>onsite_dept_managers</code> but offsite
<code>offsite_workshop_manager</code>	Same as <code>onsite_workshop_manager</code> but offsite

Info: The rule names live in `functions/index.js`. To add a new rule, add a new helper function and register it in `resolveRecipientsFromRules`.

Note — The `operator` rule is special:

- Resolves per-job using the `operatorClockNo` field — different jobs have different operators
- Ignores `isOnSite` entirely — the operator created the job and should hear about it regardless of location
- Receives a **different notification body** than other recipients: *"X minutes passed with no assignment. We've notified N people. Follow up directly."*
- `triggeredBy` on the notification doc is `stage{N}_operator_followup` (distinct from `stage{N}_escalation`) so analytics can separate the two
- Typically ticked on Stage 1 only — the operator only needs the heads-up once. If unassigned at later stages, tick it again per stage

5. The Escalation Lifecycle

Step 1 — Job is created

1. Flutter app writes a new doc to `job_cards` via `JobCard.toFirestore()`
2. The new doc has `assignedClockNos: null`, `notifiedAtStage1..4: null`, `status: "open"`, `escalationStopped: false`
3. Firestore trigger `onJobCardCreated` fires:
 - Loads config from `notification_configs/global`
 - If `job.type` is in `excluded_job_types` → returns silently (no notification)
 - Otherwise resolves `creation_recipients_by_type[jobTypeKey]` and notifies them

Step 2 — Escalation scheduler runs (every 2 minutes)

The `escalateNotifications` function executes this loop for each stage 1 → 4:

```
For each stage (1, 2, 3, 4):

- If stage.enabled is false → skip
- If stage.minutes is invalid ( $\leq 0$ ) → skip
- Query Firestore for jobs where:
  - status == "open"
  - AND notifiedAtStage<N> == null ← server-side filter
  - AND createdAt <= (now - stage.minutes)
- For each returned job:
  - If job.type is in excluded_job_types → skip
  - If job.createdAt <= stage.enabled_at → skip (pre-enable cutoff)
  - If job was assigned OR escalationStopped is true:
    - Stamp ALL unset stages at once (Stage 1..4) so the job disappears from every escalation query immediately
    - Continue (no notification sent)
  - Resolve recipients via stage.recipients_by_type[jobType] (passes job.operatorClockNo so the "operator" rule can resolve)
  - If 0 recipients resolved → log and skip (DO NOT stamp – leaves the door open if config gets fixed)
  - For each recipient:
    - If recipient.clockNo == job.operatorClockNo → use operator-specific

```

- title and body ("No response yet... follow up directly")
- Otherwise → use standard escalation title and body
- Write notifiedAtStage<N> = serverTimestamp on the job doc

Tip — Read-cost optimisation: the `notifiedAtStage<N> == null` filter is applied at the Firestore level (not in JavaScript) so already-stamped jobs are never fetched. For sites with many long-open jobs sitting around (e.g. Priority 1–3 work waiting for a press shutdown), this cuts Firestore reads by 90%+ — they only get processed once, then disappear from every future stage query.

Step 3 — Job is assigned (escalation stops)

1. Technician or manager assigns the job
2. Flutter updates `assignedClockNos`
3. `onJobCardAssigned` trigger fires → sets `escalationStopped: true`
4. Next escalation cycle sees `escalationStopped` and skips the job for all future stages

Step 4 — Technician marks "Busy"

1. Technician taps Busy on the alert
2. App writes to `alertResponses` collection
3. `onAlertResponseCreated` trigger fires → sets `escalationStopped: true` on the job
4. Creator gets a "Busy Response" notification

6. The Stamp Fields

Four timestamp fields on each job card track which stages have fired:

- `notifiedAtStage1`
- `notifiedAtStage2`
- `notifiedAtStage3`
- `notifiedAtStage4`

Value	Meaning
<code>null</code> or missing	Stage hasn't fired yet for this job
Timestamp	Stage already fired — don't fire it again

Note — Critical rule: the stamp only writes when a notification was actually sent. If recipient resolution returns 0 (e.g. nobody marked `isOnSite: true`), the stamp is **not** written so the stage will retry next cycle when conditions improve.

Tip — Assigned / busy jobs stamp all 4 stages at once. When the escalation loop encounters a job that has been assigned or has `escalationStopped: true`, it writes **every unset stamp (Stage 1–4) in a single update** instead of one stage at a time. The job disappears from every escalation query immediately, not gradually over the next hour as later stage cutoffs are reached.

If a stage is `enabled: false`, the stamp is **not** written either.

Tip — Bombardment protection: when you toggle a stage from disabled to enabled, the admin save logic writes `enabled_at = now` on that stage. The escalation loop then skips any job whose `createdAt` is before `enabled_at`. This means newly-added recipients only get notified for jobs created **after** the stage was turned on — old open jobs that pre-existed the change are ignored.

If you need to clear stamps manually (test scenarios, fixing a bad config), use the **Reset Escalation Stamps** button in Admin or call the `clearEscalationStamps` Cloud Function.

7. The Admin Settings UI

Located at: **Admin** → **Settings** → **Escalation Config**

Each stage has its own card with:

- **Switch** — toggles `enabled`
- **Trigger after (minutes)** — sets `minutes` (greyed out when disabled)

- **Recipient checkboxes** — one shared list applied to all 3 job types when saving (greyed out when disabled)
- **Validation** — if Stage 1 and Stage 2 are both enabled, Stage 1 minutes must be less than Stage 2 minutes

There are two action buttons below the cards:

- **Save Escalation Config** — writes the document to Firestore
- **Reset Escalation Stamps** — calls the `clearEscalationStamps` Cloud Function, which clears `notifiedAtStage1..4` on all open non-excluded jobs

Warning: The admin UI writes the **same recipient list** to all three job types (`mechanical` , `electrical` , `mech/elec`). If you need different recipients per job type, edit the Firestore document directly — the document structure supports it; the UI just doesn't expose that granularity yet.

8. Common Tasks

Change a stage's timing

1. Open Admin → Settings → Escalation Config
2. Change the minutes value for that stage
3. Tap Save

Disable a stage

1. Toggle the stage's Switch off in the admin screen
2. Tap Save
3. Existing stamps remain on jobs but new jobs won't escalate through that stage

Re-enable a previously disabled stage

1. Toggle the stage's Switch back on in the admin screen
2. Tap Save — a confirmation dialog warns that only jobs created from now on will trigger this stage
3. The save writes `enabled_at = now` on that stage

4. Old open jobs that existed before the toggle are **not** escalated. Only jobs created after the enable moment will trigger the stage

Backfill stamp fields on legacy job cards

After deploying new escalation fields (e.g. adding Stage 3 / Stage 4 stamps) or after enabling the read-cost optimisation, run **Admin** → **Reset Escalation Stamps** once. It writes

`notifiedAtStage1..4 = null` on every open non-excluded job, ensuring all documents are properly included in the composite indexes. New jobs created via the app already include all four fields automatically.

Add a new job type that should escalate

1. Add the new enum value to `JobType` in `lib/models/job_card.dart`
2. Add the matching key (and `recipients_by_type` entry for it on each stage) to the Firestore doc
3. Optional: add it to `jobTypeKey()` in `functions/index.js` if it needs a different Firestore key

Add a new job type that should be excluded

Add it to `excluded_job_types` in the Firestore doc, e.g.:

```
"excluded_job_types": ["maintenance", "preventative_maintenance"]
```

No code change needed.

Add a new recipient rule (e.g. "site-supervisors")

1. Write a helper function like `getOnsiteSiteSupervisors()` in `functions/index.js`
2. Add an `else if (rule === "site_supervisors")` branch in `resolveRecipientsFromRules`
3. Add the rule name to the `_allRules` list and `_ruleLabels` map in `lib/screens/admin_screen.dart` so it appears in the UI
4. Deploy functions and rebuild the app

Test the escalation

1. Make sure at least one employee has `isOnSite: true`, a valid `fcmToken`, and a position matching the recipient rule (e.g. "Manager" for `onsite_managers`)

2. Create a new mechanical job — leave it unassigned
3. Wait 5 minutes (or the configured Stage 1 minutes)
4. Watch the `escalateNotifications` Cloud Function logs
5. You should see: `Stage 1 (5min): found N jobs` → `recipients=M` → notification sent

9. Troubleshooting

Symptom	Likely cause
No escalation logs at all	Cloud Scheduler not invoking the function — check it's deployed in <code>europa-west1</code>
<code>Stage X: disabled, skipping</code>	The <code>enabled</code> flag is <code>false</code> in the config
<code>found N jobs</code> but <code>recipients=0</code>	No employees match the rule (e.g. nobody onsite, no managers in matching dept)
<code>recipients=N</code> but no notifications received	Stale FCM tokens — the function auto-clears them on <code>messaging/registration-token-not-registered</code>
Same person notified twice at the same stage	Job has two matching rules resolving to the same person — <code>resolveRecipientsFromRules</code> deduplicates by <code>clockNo</code> so this shouldn't happen
Jobs marked "already notified" but nobody got the notification	Old buggy data — use Reset Escalation Stamps to clear
A stage finds 0 jobs even though there are obviously old open jobs	Composite index doesn't include those documents because <code>notifiedAtStage<N></code> is missing (legacy data). Run Reset Escalation Stamps once to backfill the fields to <code>null</code> .
Read counts in Firebase Console suddenly spike	Either (a) the <code>notifiedAtStage<N> == null</code> filter was removed from the query, or (b) the composite index isn't built yet after a deploy. Check the indexes tab in Firebase Console.

10. File Map

File	What lives there
<code>functions/index.js</code>	All Cloud Functions: <code>escalateNotifications</code> , <code>onJobCardCreated</code> , <code>onJobCardAssigned</code> , <code>onAlertResponseCreated</code> , <code>clearEscalationStamps</code> , plus recipient resolver helpers
<code>lib/models/job_card.dart</code>	<code>JobCard</code> model with <code>notifiedAtStage1..4</code> fields
<code>lib/screens/admin_screen.dart</code>	The Escalation Config UI and Reset Escalation Stamps button
<code>lib/services/firestore_service.dart</code>	<code>getNotificationConfig()</code> and <code>saveNotificationConfig()</code>
<code>firestore.indexes.json</code>	Composite indexes — including <code>status + createdAt</code> used by escalation queries

11. Hard Rules — Don't Break These

1. The schedulers must remain in `europa-west1`. `escalateNotifications` and `autoCloseMonitoringJobs` must stay in `europa-west1`. Moving them to `africa-south1` will break the Cloud Scheduler binding.

2. Stamps only write when notifications are actually sent. If you add a new stage, make sure 0-recipient cases don't write the stamp — otherwise jobs get permanently marked as "already notified" without anyone being told.

3. `createdAt` must always be the last field in any composite Firestore index that uses it with a range filter (`<=` , `>=`). Other fields are equality and come first.

4. Don't combine multiple null-equality filters with a range filter in one Firestore query. The current escalation query uses *one* null filter (`notifiedAtStageN == null`) plus a range on `createdAt` , which works fine with the correct composite index. Adding a second null filter

(e.g. `assignedClockNos == null`) silently returns 0 results — filter that one in JavaScript after the fetch instead.

5. Composite indexes don't include documents with missing fields. If a job card was created before `notifiedAtStage3` / `notifiedAtStage4` existed on the model, those documents won't appear in the Stage 3 / Stage 4 queries. Run **Admin** → **Reset Escalation Stamps** once after deploying field additions — it explicitly writes `null` on every open non-excluded job, putting them all in the index.

CTP Job Cards · Notification Escalation System Reference · Engineering documentation

Accuracy reconciled (2026-05-18)

The source HTML was transcribed faithfully and then cross-checked against `functions/index.js` `defaultNotificationConfig()`. The authoritative defaults are:

Stage	Default time	Default state
Stage 1	5 minutes	Enabled
Stage 2	10 minutes	Enabled
Stage 3	30 minutes	Disabled (admin can enable)
Stage 4	60 minutes	Disabled (admin can enable)

The HTML correctly states that Stages 3 and 4 are disabled by default but does not surface their 30 / 60-minute default timings. Treat the table above as canonical; the next regeneration of this doc should be the source of truth for the HTML.

Scheduler cadence (`escalateNotifications` runs every 2 minutes in `europa-west1`) and validation rules (Stage N minutes must be less than Stage N+1 minutes) are both confirmed against the code.